Java Records

The cleanest way to create DTOs
(Java 16+)

ALAN BIJU

What Are Java Records?

Java Records are a special type of class introduced to represent **immutable data.**

They are designed for:

- Data transfer objects (DTOs)
- API requests & responses
- Value-based objects

📌 Introduced as preview in Java 14
📌 Stable from Java 16

Traditional DTO vs Record

❌ **Traditional DTO**

```
class UserDto {
    private Long id;
    private String name;

    // constructor
    // getters
    // equals & hashCode
}
```

✅ **Record**

```
public record UserDto(Long id, String name) {}
```

✔️ Same functionality
✔️ 80% less code
✔️ Better readability

What Java Generates Automatically

When you create a record, Java auto-generates:

- private final fields
- Canonical constructor
- Public accessor methods
- `equals()`
- `hashCode()`
- `toString()`

📌 You **cannot override** this behavior accidentally
📌 Guarantees consistency

Immutability by Default

All record fields are:

- `final`
- Set only via constructor

```
UserDto user = new UserDto(1L, "Alan");
// user.name = "New Name"; ❌ NOT allowed
```

✔️ Thread-safe by design
✔️ Ideal for concurrent applications
✔️ Fewer bugs

Records in Spring Boot (REST APIs)

Records are perfect for API contracts.

```
@GetMapping("/users/{id}")
public UserDto getUser(@PathVariable Long id) {
    return new UserDto(id, "Alan");
}
```

Request Payloads with Records

Records also work well for **request bodies.**

```
@PostMapping("/users")
public void createUser(@RequestBody UserDto dto) {
    // use dto.id(), dto.name()
}
```

Validation with Records

Bean Validation works perfectly with records.

```
public record UserDto(
    @NotNull Long id,
    @NotBlank String name,
    @Email String email
) {}
```

Custom Logic in Records

Records are not just passive data holders.

You can add:

- Custom methods
- Compact constructors

```
public record UserDto(String email) {
    public UserDto {
        if (!email.contains("@")) {
            throw new IllegalArgumentException
                ("Invalid email");
        }
    }
}
```

Records + Modern Java Features

Records work beautifully with:

- Pattern matching
- Modern switch
- Sealed classes

```
switch (obj) {
    case UserDto(Long id, String name) -> ...
}
```

Best Practices

✓ Use records for DTOs
✓ Keep records small
✓ Validate in constructor
✗ Don’t add complex logic
✗ Don’t use for persistence

> 👉 If your class **only carries data,**
> 👉 **Use a Record**

Java Records reduce code and sharpen design thinking.

IF YOU FIND THIS HELPFUL, LIKE AND SHARE IT WITH YOUR FRIENDS

ALAN BIJU | @alanbiju98